CS 6515: Graduate Algorithms

Lecture 7: Prim's Algorithm and Matchings

Instructors: Sahil Singla and Gerandy Brito

Fall 2026

1 Announcements

1. HW2 is due Sep 17.
2. Exam 1 is Sep 22. Syllabus Lectures 1-8.

2 Recap: the MST cut property

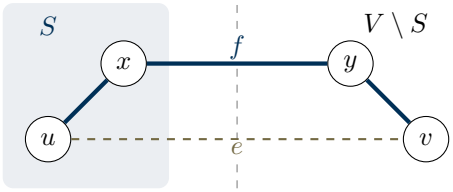
Let $G = (V, E)$ be a connected, undirected graph with edge weights $w(e)$. Write $n = |V|$ and $m = |E|$. A *spanning tree* connects all vertices without a cycle, so it has exactly $n - 1$ edges. Our goal is to find a spanning tree T minimizing $w(T) = \sum_{e \in T} w(e)$.

With nonnegative weights, some cheapest connected spanning subgraph is a tree: repeatedly delete a cycle edge without increasing cost. A minimum spanning tree need not give shortest routes from a root; its objective is the *total edge weight*.

Definition (cut). For $\varnothing \neq S \subsetneq V$, the cut $(S, V \setminus S)$ consists of all edges with one endpoint in S and the other outside S. A light edge of this cut is an edge of minimum weight among its crossing edges.

Theorem (cut property). For any nontrivial cut, any light edge e belongs to *some* MST. If e is the unique light edge of that cut, it belongs to *every* MST.

Proof plan. Start with an MST that omits e. Adding e creates a cycle; deleting an appropriate edge from that cycle restores a tree at no greater cost.



Schematic: the solid path is in T^*; adding the dashed edge e closes a cycle. The path must cross the cut on another edge f.

Exchange proof. Let T^* be an MST. If $e \in T^*$, we are done. Otherwise, $T^* \cup \{e\}$ contains one cycle. Since e crosses the cut, the rest of the cycle must cross back on an edge $f \in T^*$. Because e is light, $w(e) \leq w(f)$. Consequently,

$$T' = T^* \cup \{e\} \setminus \{f\}, \qquad w(T') = w(T^*) + w(e) - w(f) \leq w(T^*).$$

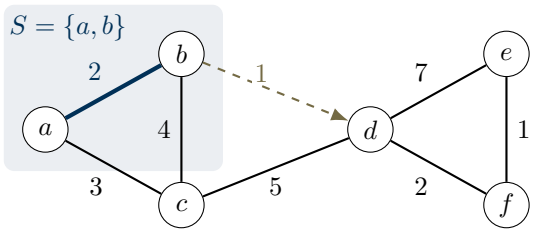
Deleting a cycle edge preserves connectivity, so T' is a spanning tree. Optimality of T^* implies that T' is also an MST, now containing e. If e is uniquely light, the inequality would be strict; hence no MST can omit it. □

3 Prim's algorithm

Let G be connected and undirected. Prim's algorithm grows one tree from a chosen root r.

Ties and compatible choices. In an equal-weight triangle, each edge is light for a cut but no tree contains all three. Thus "some MST" cannot be replaced by "every MST" when there are ties. Prim chooses cuts whose light edges are compatible with all earlier choices.

1. Start with the visited set $S = \{r\}$ and $T = \varnothing$.
2. Choose a minimum-weight edge (u, v) crossing $(S, V \setminus S)$, with $u \in S$ and $v \notin S$.
3. Add (u, v) to T, add v to S, and repeat until $S = V$.



Starting at a, Prim first selects (a, b). The crossing edges now have weights $3, 4, 1$, so the next edge is (b, d) (dashed).

Proof plan and invariant. We must show that *some MST contains all edges selected so far*, not just that each selected edge belongs to some MST on its own.

Correctness. Initially $T = \varnothing$, so the invariant holds. Suppose an MST T^* contains T, and let e be the next minimum-weight edge across $(S, V \setminus S)$. If $e \notin T^*$, adding it creates a cycle. That cycle has another crossing edge f, with $w(f) \geq w(e)$. No edge of T crosses this cut, so $f \notin T$. Replacing f by e gives an MST containing $T \cup \{e\}$. Thus the invariant is preserved. At the end T has $n - 1$ edges, so it is itself that MST. □

3.1 Priority-queue implementation

For every unvisited vertex v, maintain

$$\text{key}[v] = \min\{w(u, v) : u \in S,\ (u, v) \in E\},$$

where the minimum of an empty set is $+\infty$. Store a parent attaining any finite minimum. Initialize these keys by scanning the edges out of r. A min-priority queue returns the unvisited vertex with the smallest key. When a vertex u is added, scan its incident edges and decrease the keys of its unvisited neighbors when possible.

With adjacency lists and a binary heap, Prim's algorithm takes

$$\mathcal{O}((n + m) \log n) = \mathcal{O}(m \log n)$$

time for a connected graph.

As with Dijkstra, there are at most n extract-min operations and m decrease-key operations. Fibonacci heaps give $\mathcal{O}(m + n \log n)$ time.

Compare with Dijkstra. The queue operations look the same, but the keys mean different things:

	Meaning of key at v	Candidate when scanning (u, v)
Dijkstra	Length of a path from s	$d[u] + w(u, v)$
Prim	Cost of one edge from S	$w(u, v)$

An MST need not be a shortest-path tree. Prim's exchange proof also works with negative edge weights, unlike Dijkstra's proof. Here an MST always means a minimum-weight *spanning tree*.

Exercise. Continue Prim's algorithm on the displayed graph. Record the visited set, keys, and selected edge after every iteration. Can different tie-breaking choices produce different MSTs?

4 Matchings

Let $G = (V, E)$ be an undirected graph.

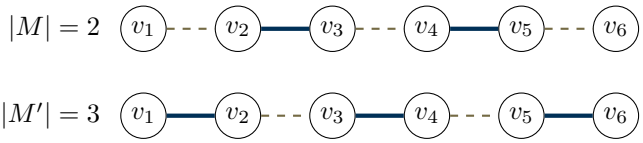
Definition (matching). A matching is a set $M \subseteq E$ such that no vertex is incident to more than one edge of M.

Goal. Find a matching of maximum cardinality.

A matching is *maximal* if no edge can be added to it, while it is *maximum* if no larger matching exists. These words are not equivalent: a greedy matching can be maximal but smaller than a maximum matching.

Matching models one-to-one assignments, including courses to classrooms, donors to recipients, and participants on two sides of a marketplace.

Proof plan. Adding one edge may be impossible because its endpoints are already matched. Instead, look for a chain of reassignments joining two unmatched endpoints and gaining one edge overall. We will prove that if no such improvement exists, the matching is maximum.



Solid blue edges form the current matching; dashed edges are outside it. The upper matching is maximal but not maximum. Flipping all five path edges gives the lower matching.

Definition (augmenting path). Given a matching M, an augmenting path is a simple path whose edges alternate between $E \setminus M$ and M, and whose two endpoints are unmatched by M.

An augmenting path begins and ends with unmatched edges. Therefore it contains one more unmatched edge than matched edge.

Observation. If P is an augmenting path, then

$$M \triangle E(P) = \big(M \setminus E(P)\big) \cup \big(E(P) \setminus M\big)$$

is a matching of size $|M| + 1$. We say that we *augment along* P.

Every internal vertex of P loses one matched edge and gains one; each endpoint gains its first matched edge. All other vertices are unchanged. This proves feasibility as well as the increase in size.

Exercise. In the displayed example, why can no single edge be added to the upper matching? Why does flipping the entire path still improve it?

5 When is a matching maximum?

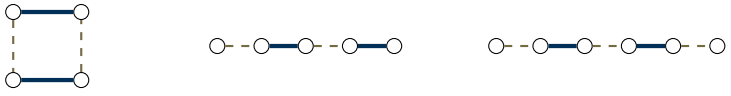
Theorem (Berge's augmenting-path theorem). A matching M is maximum if and only if there is no augmenting path with respect to M.

One direction follows from the preceding observation: an augmenting path proves that M is not maximum.

For the converse, suppose a larger matching M^* exists. Consider the symmetric difference

$$M \triangle M^*.$$

Every vertex has degree at most two in this subgraph, so every component with an edge is a path or a cycle. Along each component, edges alternate between M and M^*.



cycle
2 from each

even path
2 from each

augmenting path for M
3 from M^*, 2 from M

Components of $M \triangle M^*$: solid blue denotes $M \setminus M^*$; dashed gold denotes $M^* \setminus M$. An odd path can instead favor M; the total size difference guarantees that some path favors M^*.

An alternating cycle contains equally many edges from the two matchings. So does an even alternating path. Since $|M^*| > |M|$, some component must contain one more M^* edge than M edge. That component is a path beginning and ending with M^* edges; its endpoints are unmatched by M. It is therefore an augmenting path for M, a contradiction. □

To justify the endpoint claim: an endpoint cannot have another M edge in the symmetric difference, or it would not be an endpoint. It cannot have an edge common to M and M^* either, because it already has an incident M^* edge in this path.

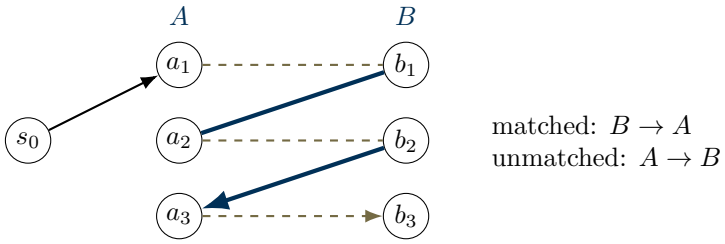
Algorithmic plan. Start with $M = \varnothing$. Repeatedly find an augmenting path and augment. If none exists, the theorem certifies that M is maximum. The remaining issue is how to find such a path efficiently.

General graphs. Berge's theorem applies to all undirected graphs. Finding an augmenting path in a general graph is more subtle because of odd cycles; Edmonds' blossom algorithm handles this case in polynomial time. We focus on the simpler bipartite case.

6 Bipartite graphs

Definition (bipartite graph). A graph is bipartite if $V = A \cup B$, $A \cap B = \varnothing$, and every edge has one endpoint in A and the other in B.

For a current matching M, orient every unmatched edge from A to B and every matched edge from B to A. Add a super-source connected to unmatched vertices of A, and search for a directed path to an unmatched vertex of B. Such a path is exactly an augmenting path.



Here $M = \{(a_2, b_1), (a_3, b_2)\}$. Search from the only unmatched left vertex a_1 reaches the unmatched right vertex b_3 via $a_1, b_1, a_2, b_2, a_3, b_3$. Flipping gives a matching of size 3.

This is the same six-vertex path as before, drawn with its bipartition visible. BFS from Lecture 6 supplies the search; parent pointers recover the actual path. After flipping, reverse the orientations of its edges and update which vertices are unmatched before the next search.

A BFS or DFS finds one in $\mathcal{O}(n+m)$ time. Since each augmentation increases the matching size by one and a matching has at most $n/2$ edges, this yields a simple polynomial-time algorithm using

$$\mathcal{O}(n(n+m))$$

time. Faster algorithms can augment along several paths at once.

Exercise. Prove the correspondence between directed paths in the oriented graph and augmenting paths. Why does this straightforward orientation argument become more subtle in a nonbipartite graph?

Bridge to max flow. The alternating-edge idea is a special case of residual capacity. Next time we develop maximum flow and then recover bipartite matching as an application.